面向对象分析与设计

Object Oriented Analysis and Design

——产品模块设计

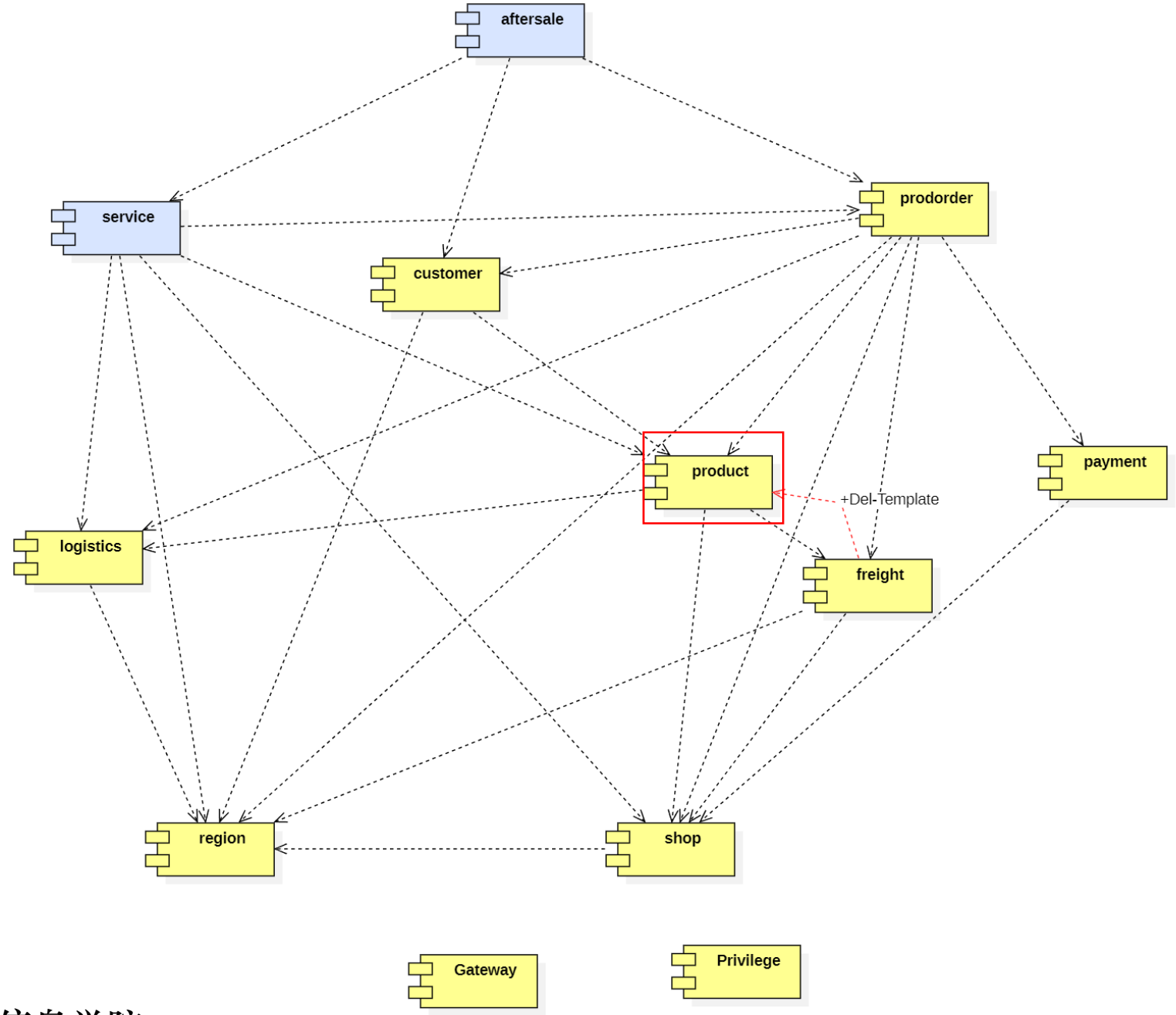
Design of Product Catalog Module

邱明 博士

厦门大学信息学院

mingqiu@xmu.edu.cn

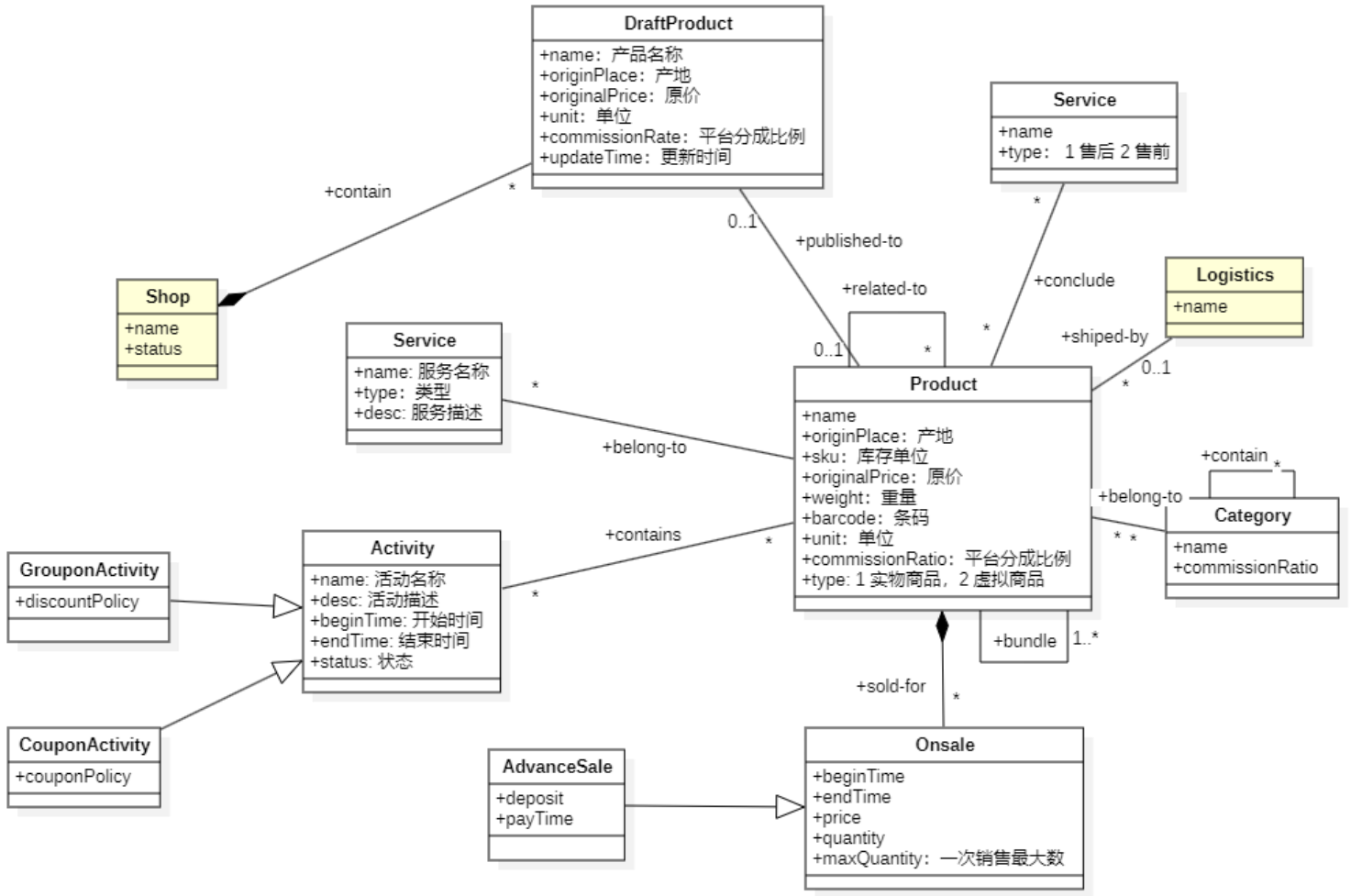
2024年秋季学期


aftersale
service
prodorder
customer
product
payment
+Del-Template
logistics
freight
region
shop
Gateway
Privilege

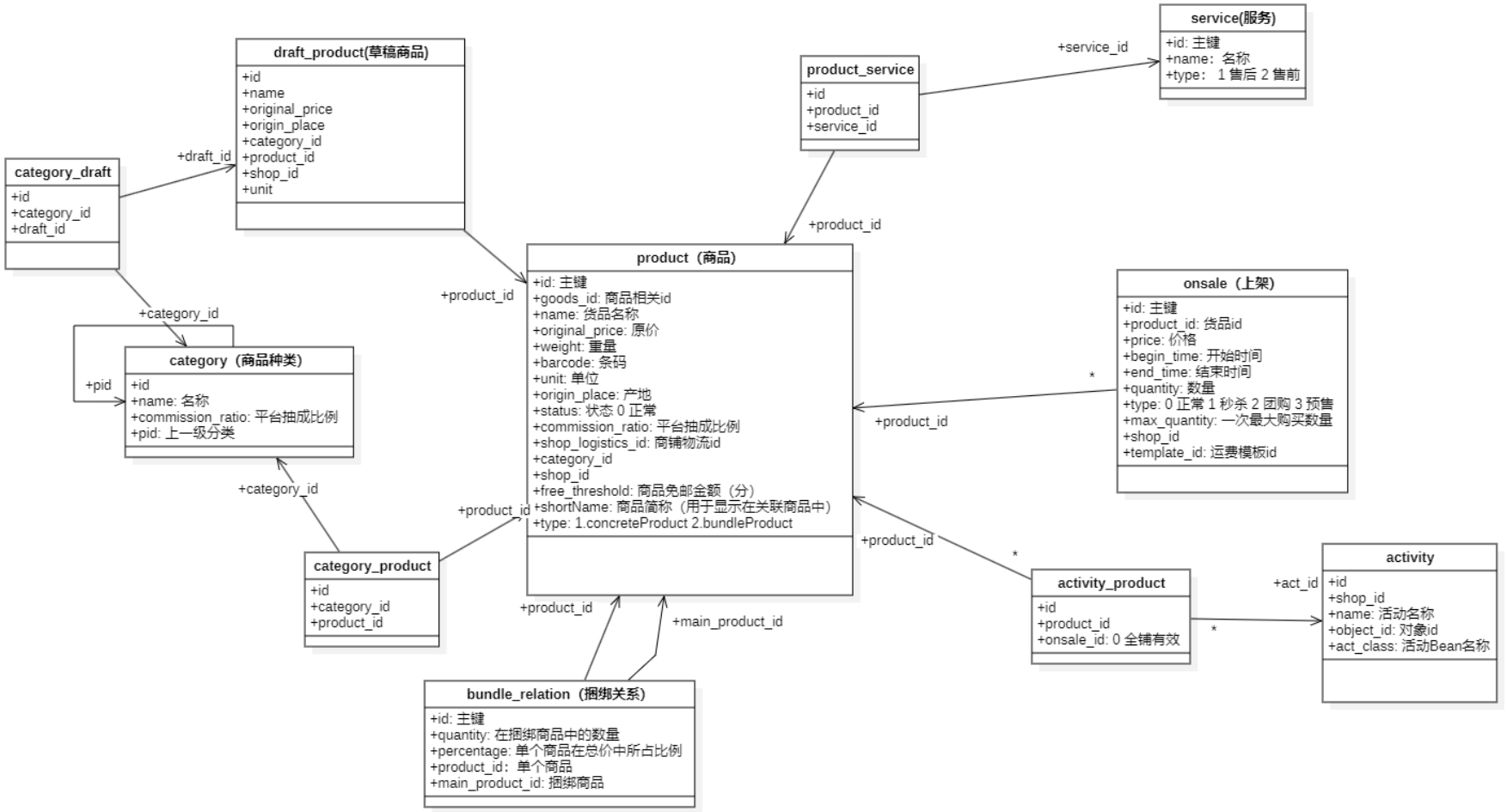
1. 静态模型设计

Static Model Design

1.1 领域模型
Domain Model

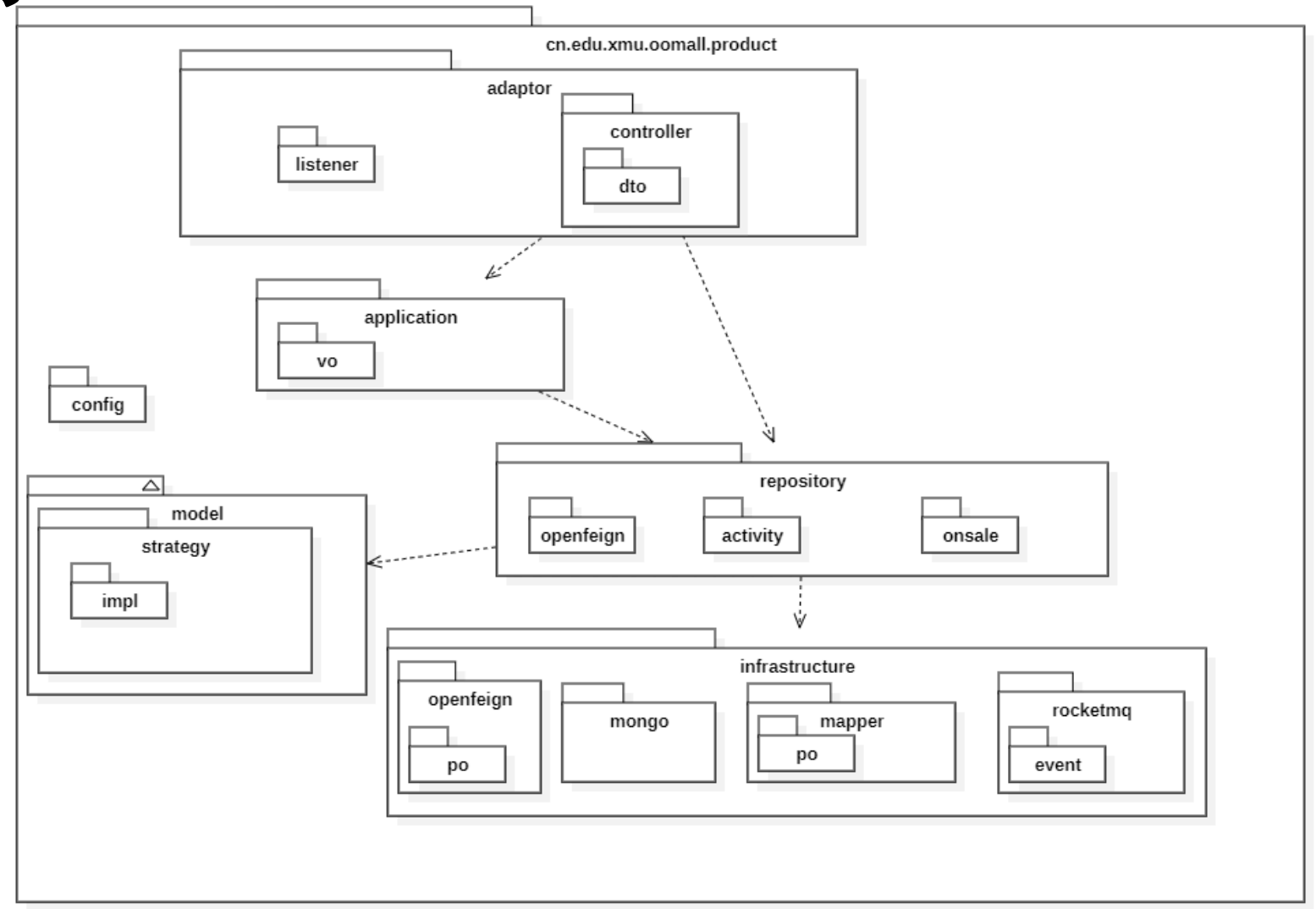


1.2 数据库设计
Database Design

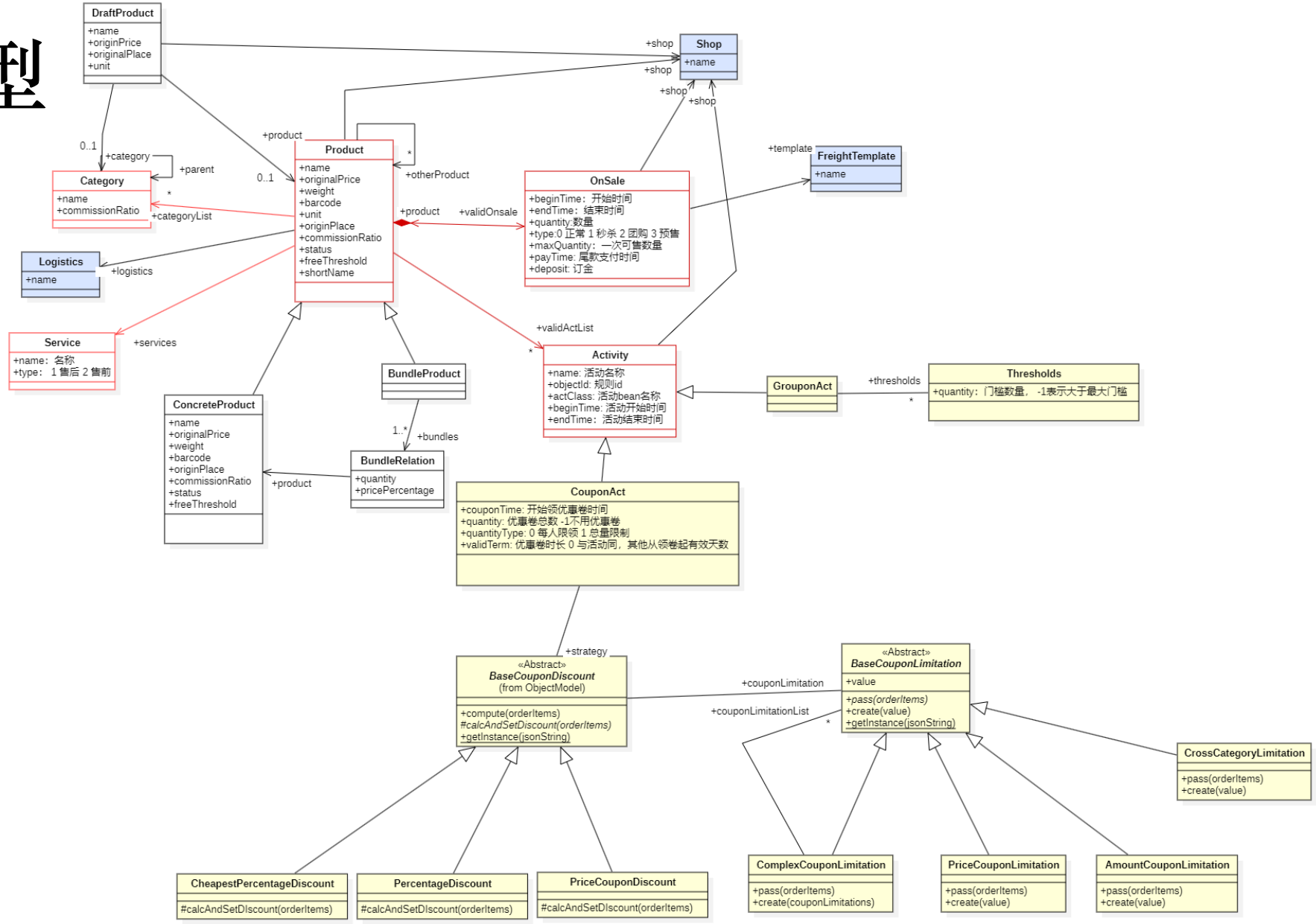


1.3 包结构

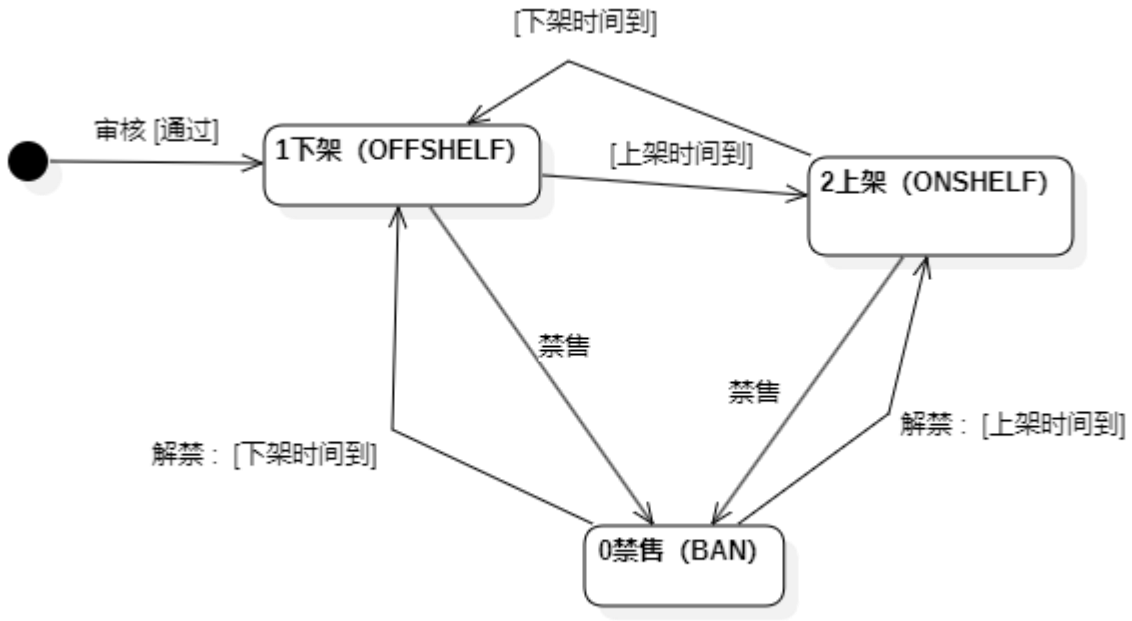
Package



1.4 对象模型
Object Model

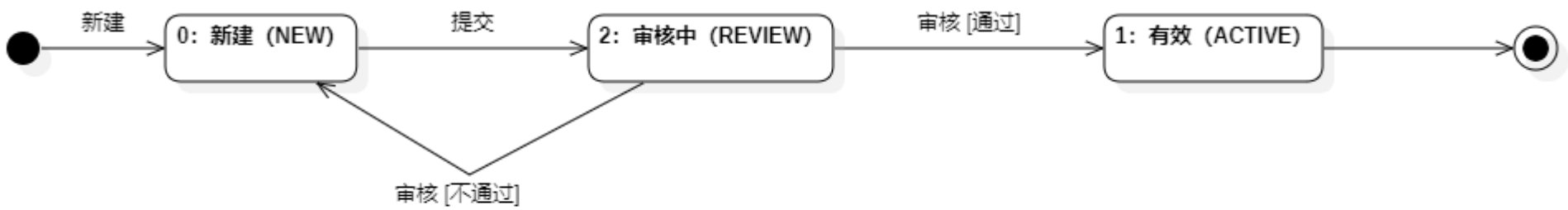


1.4 对象模型
Object Model



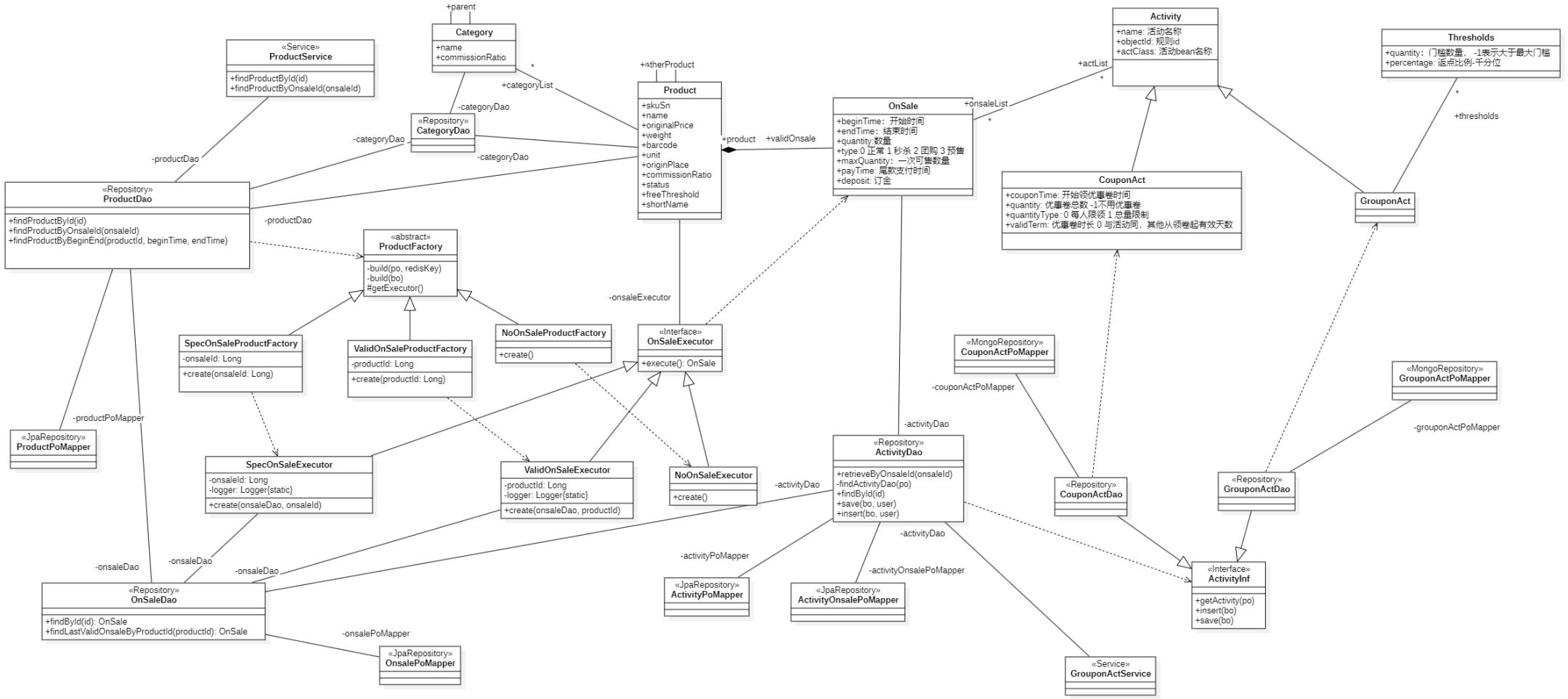
产品对象状态

1.4 对象模型
Object Model



活动对象状态

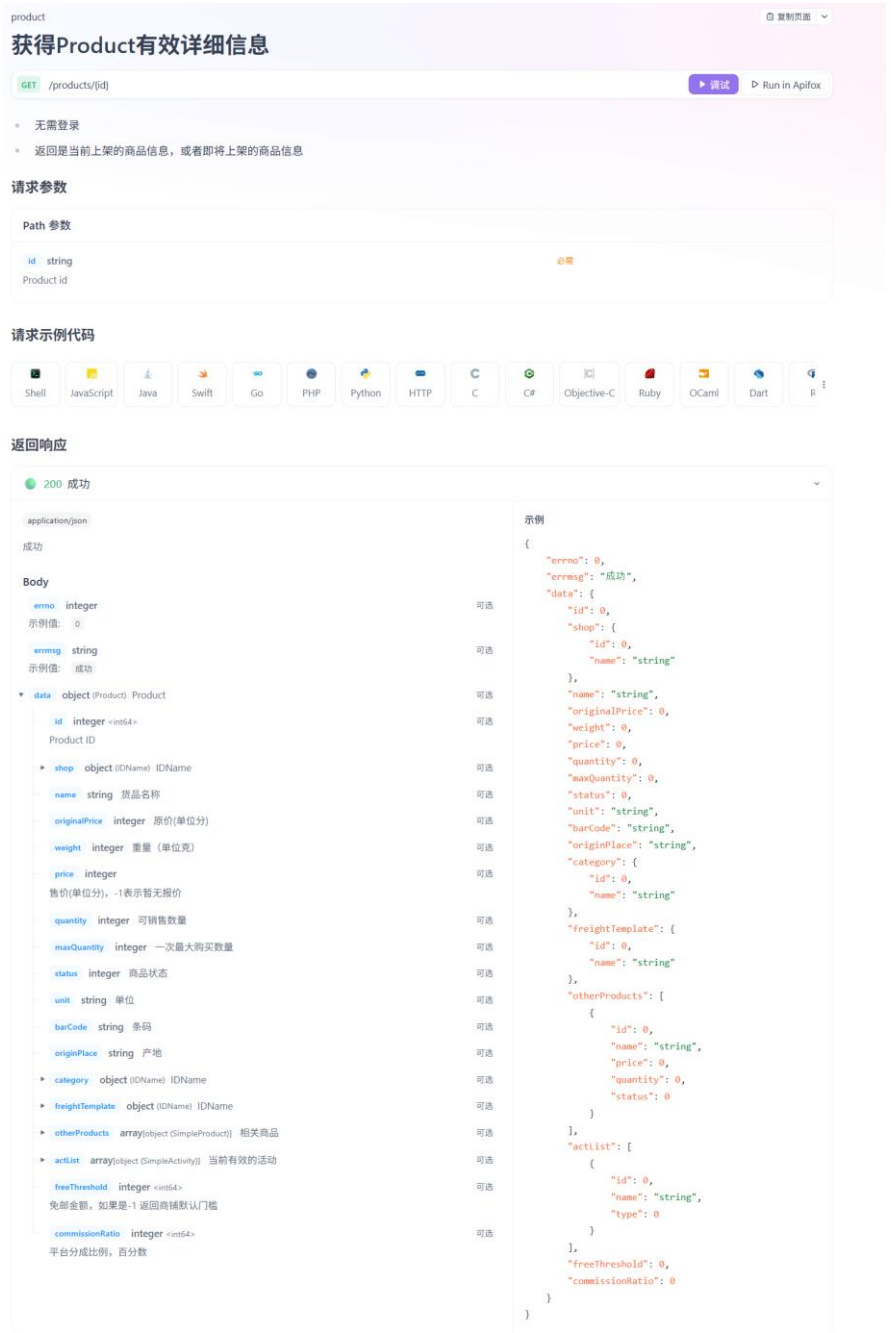
1.5 静态模型
Static Model



2. 动态模型设计

Dynamic Model Design

2.1 顾客查看商品详细信息



product

获得Product有效详细信息

GET /products/{id}

调试 Run in Apifox

- 无需登录
- 返回是当前上架的商品信息，或者即将上架的商品信息

请求参数

Path 参数

id string 必需

Product id

请求示例代码

Shell JavaScript Java Swift Go PHP Python HTTP C C# Objective-C Ruby OCaml Dart R

返回响应

200 成功

application/json

成功

Body

errno integer 可选

示例值: 0

errmsg string 可选

示例值: 成功

data object (Product) Product 可选

id integer <int64> 可选

Product ID

shop object (IDName) IDName 可选

name string 货品名称 可选

originalPrice integer 原价(单位分) 可选

weight integer 重量（单位克） 可选

price integer 可选

售价(单位分)，-1表示暂无报价

quantity integer 可销售数量 可选

maxQuantity integer 一次最大购买数量 可选

status integer 商品状态 可选

unit string 单位 可选

barCode string 条码 可选

originPlace string 产地 可选

category object (IDName) IDName 可选

freightTemplate object (IDName) IDName 可选

otherProducts array[object (SimpleProduct)] 相关商品 可选

actList array[object (SimpleActivity)] 当前有效的活动 可选

freeThreshold integer <int64> 可选

免邮金额，如果是-1 返回商铺默认门槛

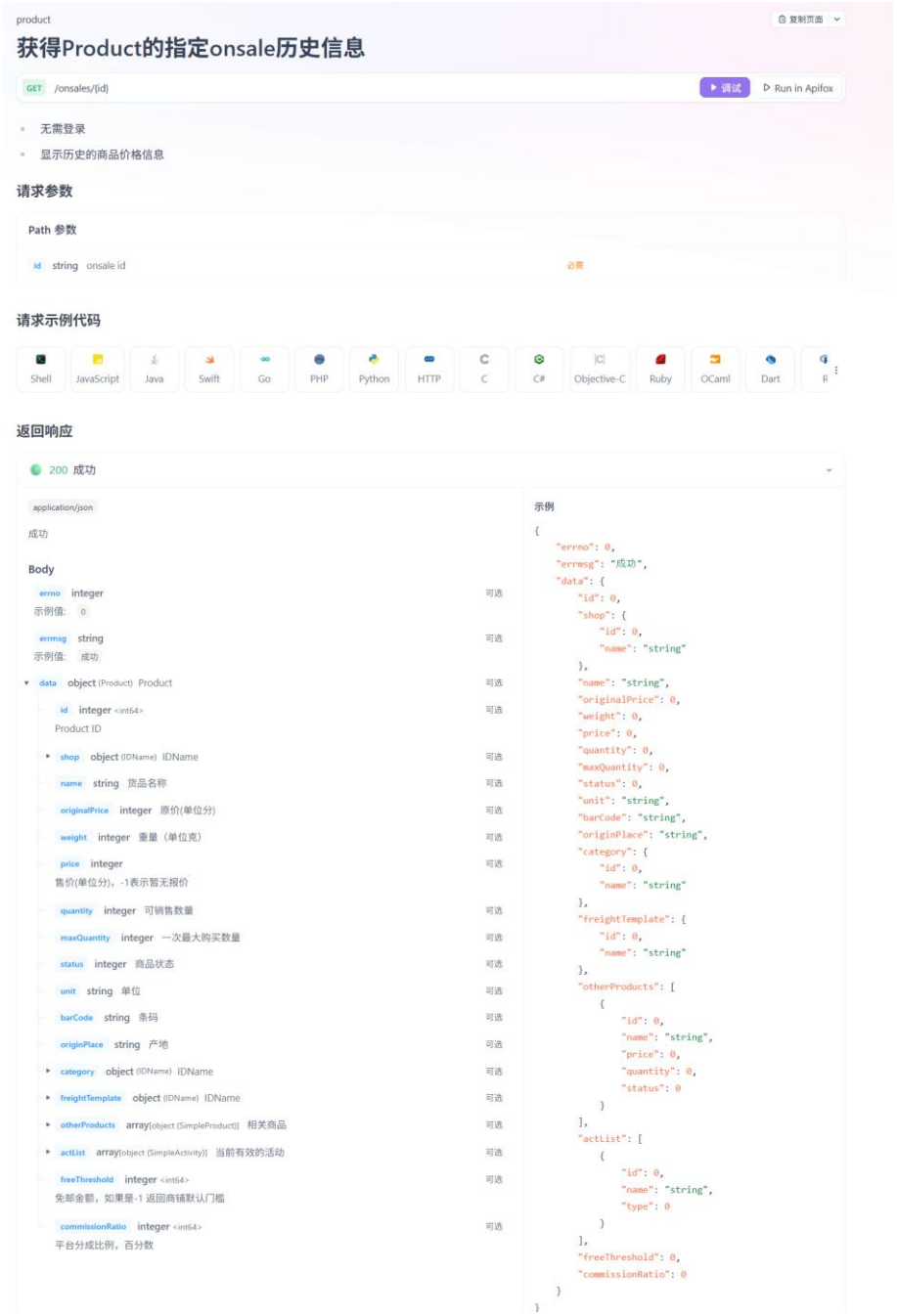
commissionRatio integer <int64> 可选

平台分成比例，百分数

示例

```
{
    "errno": 0,
    "errmsg": "成功",
    "data": {
        "id": 0,
        "shop": {
            "id": 0,
            "name": "string"
        },
        "name": "string",
        "originalPrice": 0,
        "weight": 0,
        "price": 0,
        "quantity": 0,
        "maxQuantity": 0,
        "status": 0,
        "unit": "string",
        "barCode": "string",
        "originPlace": "string",
        "category": {
            "id": 0,
            "name": "string"
        },
        "freightTemplate": {
            "id": 0,
            "name": "string"
        },
        "otherProducts": [
            {
                "id": 0,
                "name": "string",
                "price": 0,
                "quantity": 0,
                "status": 0
            }
        ],
        "actList": [
            {
                "id": 0,
                "name": "string",
                "type": 0
            }
        ],
        "freeThreshold": 0,
        "commissionRatio": 0
    }
}
```

2.1 顾客查看商品详细信息



https://qbcgzddsjf.apifox.cn/361366745e0

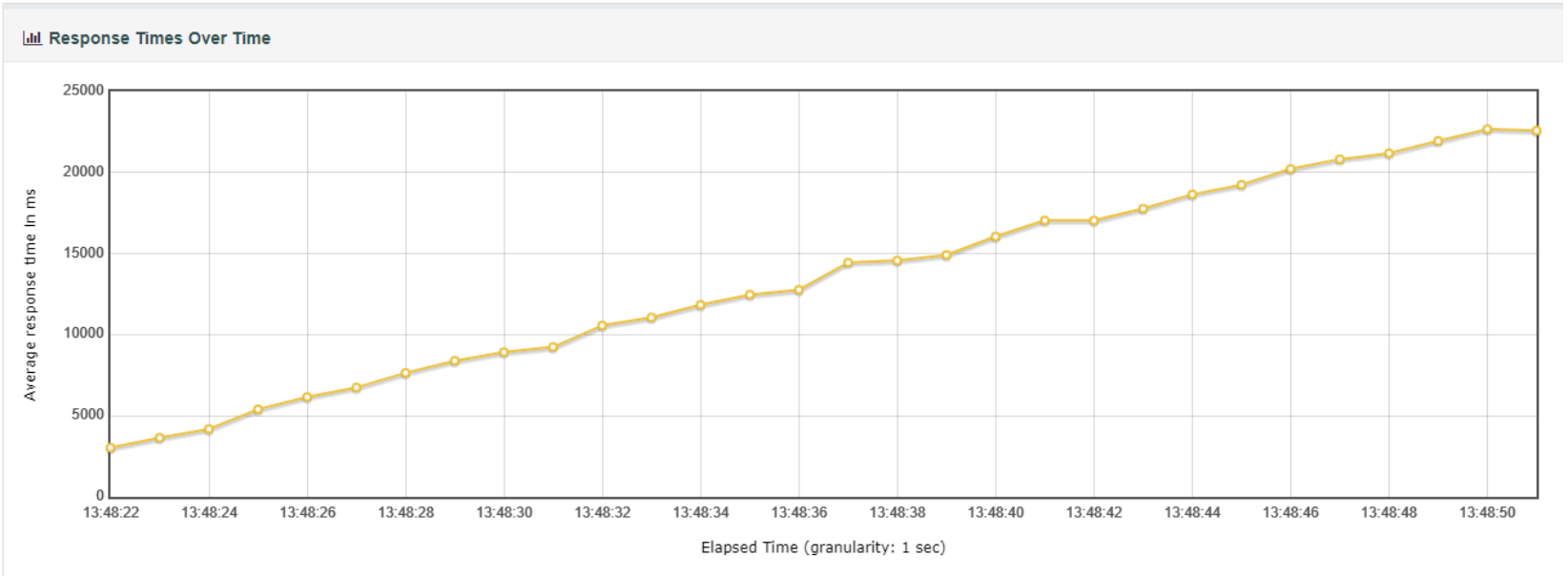
2.1 顾客查看商品详细信息

- 两个API有相似性，需要尽量重用代码
- 信息来自于对象模型中多个对象
  - 需要多次查表
  - 而且在大负载的情况下，性能衰减严重

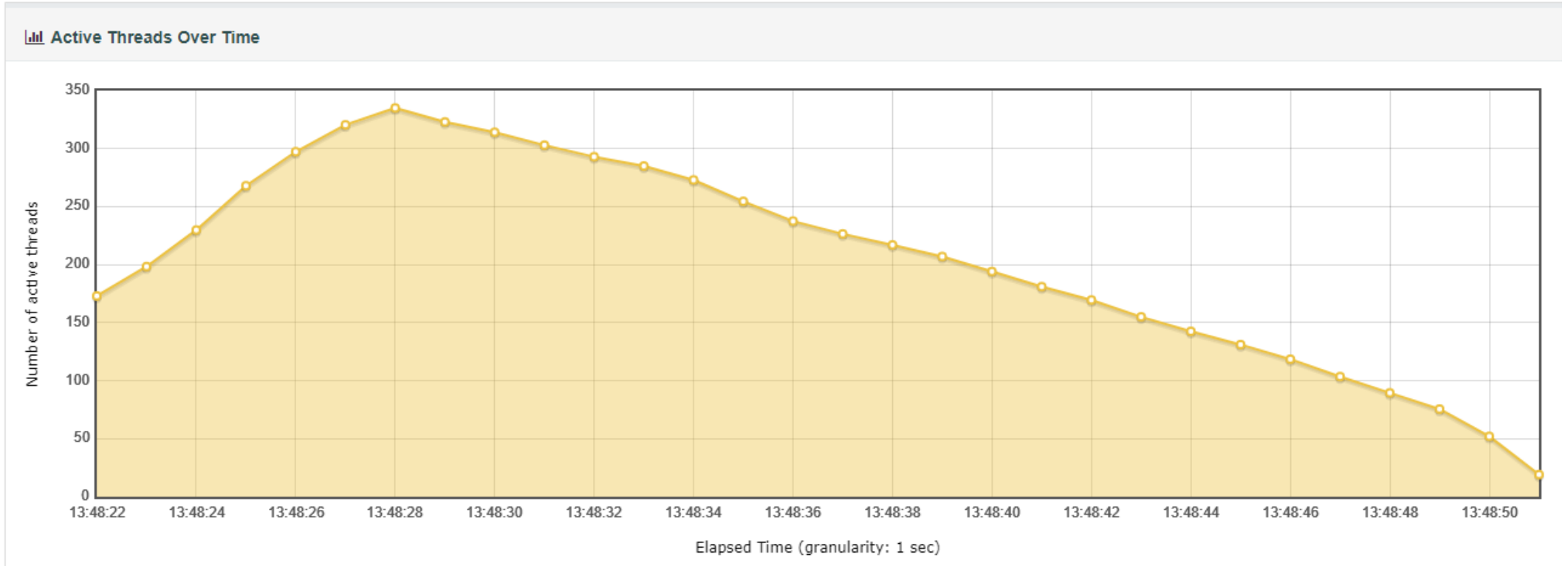
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**400**	**4**	**1.00%**	**15050.78**	**2931**	**23888**	**15480.50**	**22527.80**	**22887.70**	**23690.16**	**12.54**	**11.47**	**1.63**
redis	400	4	1.00%	15050.78	2931	23888	15480.50	22527.80	22887.70	23690.16	12.54	11.47	1.63

2.1 顾客查看商品详细信息

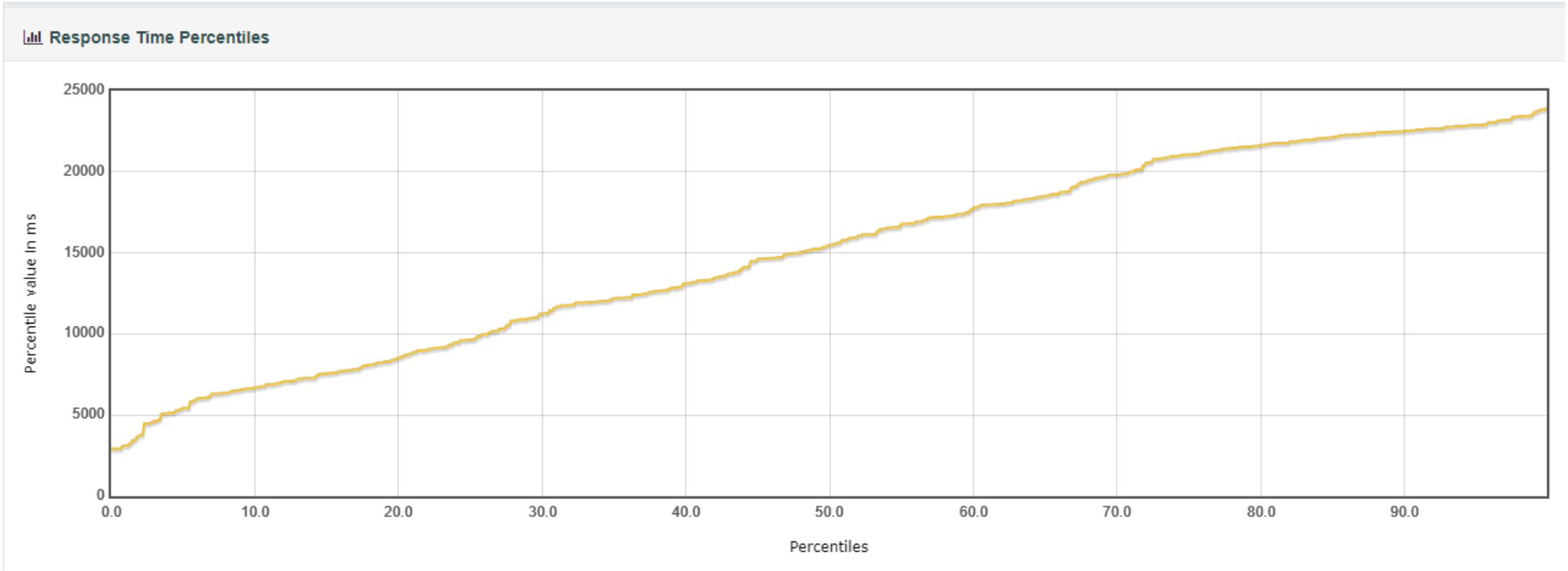
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**400**	**4**	**1.00%**	**15050.78**	**2931**	**23888**	**15480.50**	**22527.80**	**22887.70**	**23690.16**	**12.54**	**11.47**	**1.63**
redis	400	4	1.00%	15050.78	2931	23888	15480.50	22527.80	22887.70	23690.16	12.54	11.47	1.63



2.1 顾客查看商品详细信息



2.1 顾客查看商品详细信息

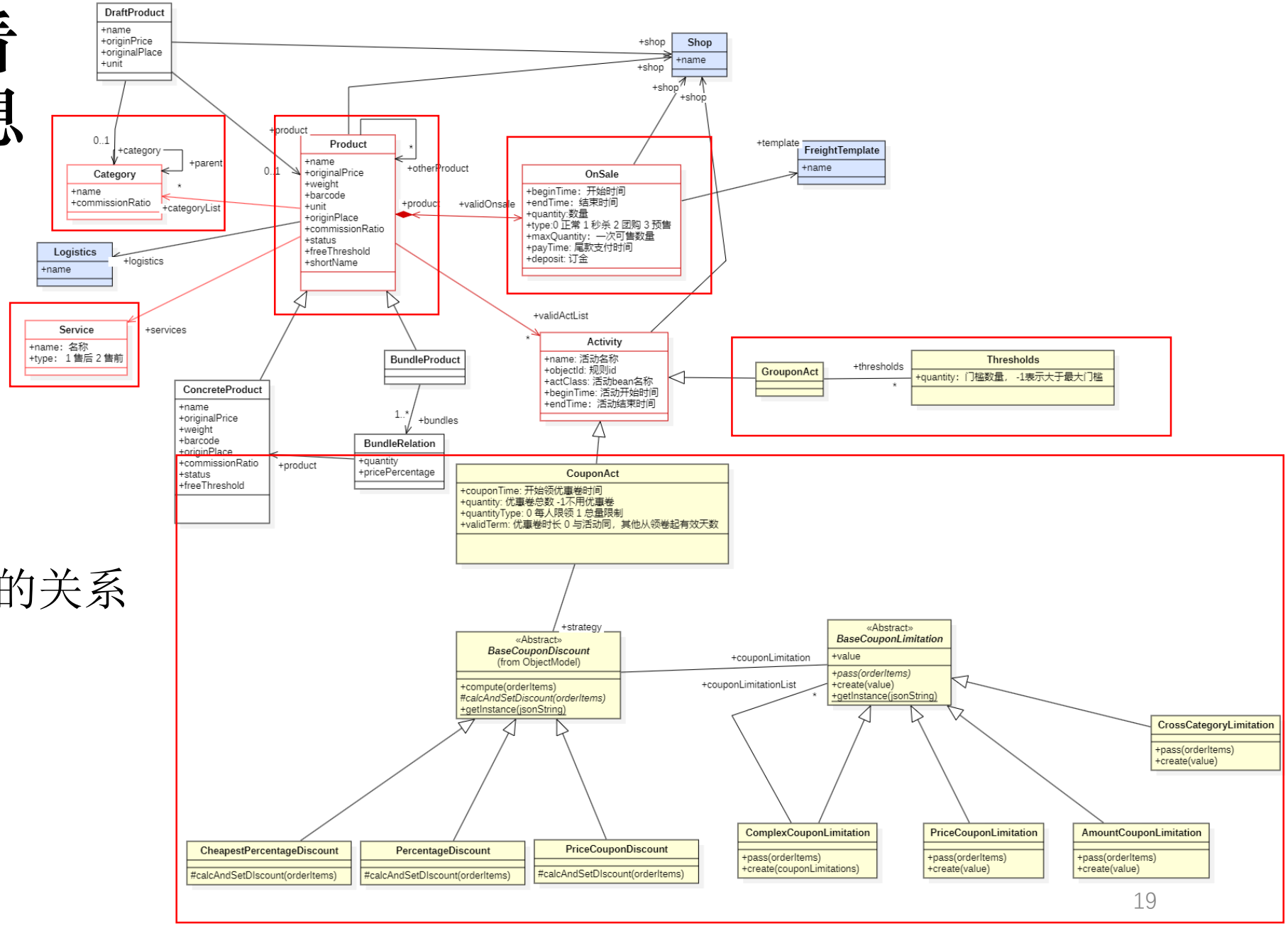


2.1 顾客查看商品详细信息

- 解决的办法
  - 利用Redis缓存，将重复查找的数据改由缓存读取
  - 缓存po，bo还是vo？
  - 关系如何缓存？

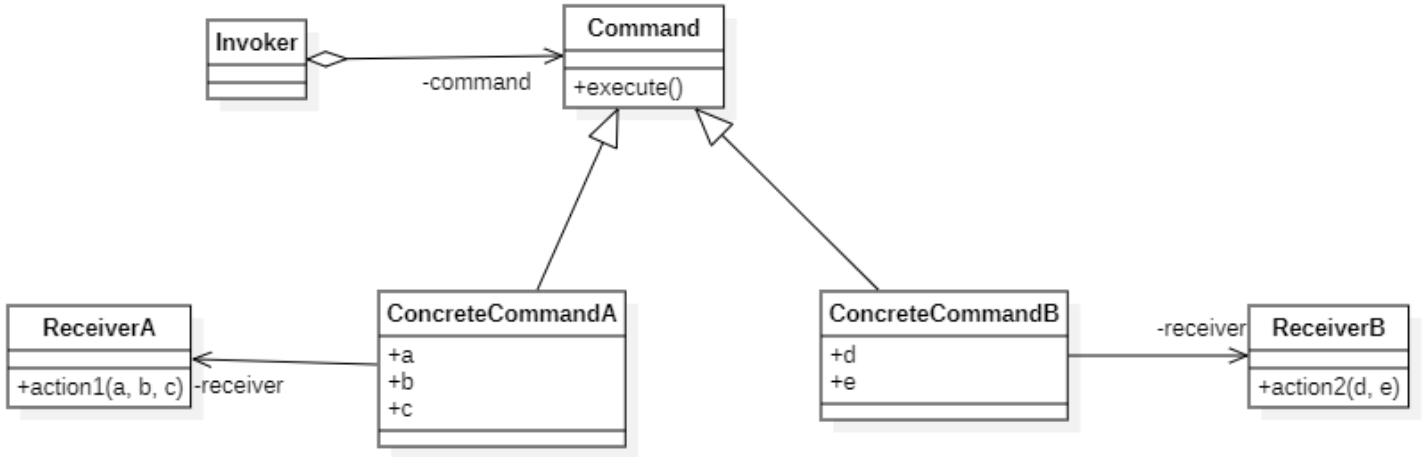
2.1 顾客查看商品详细信息

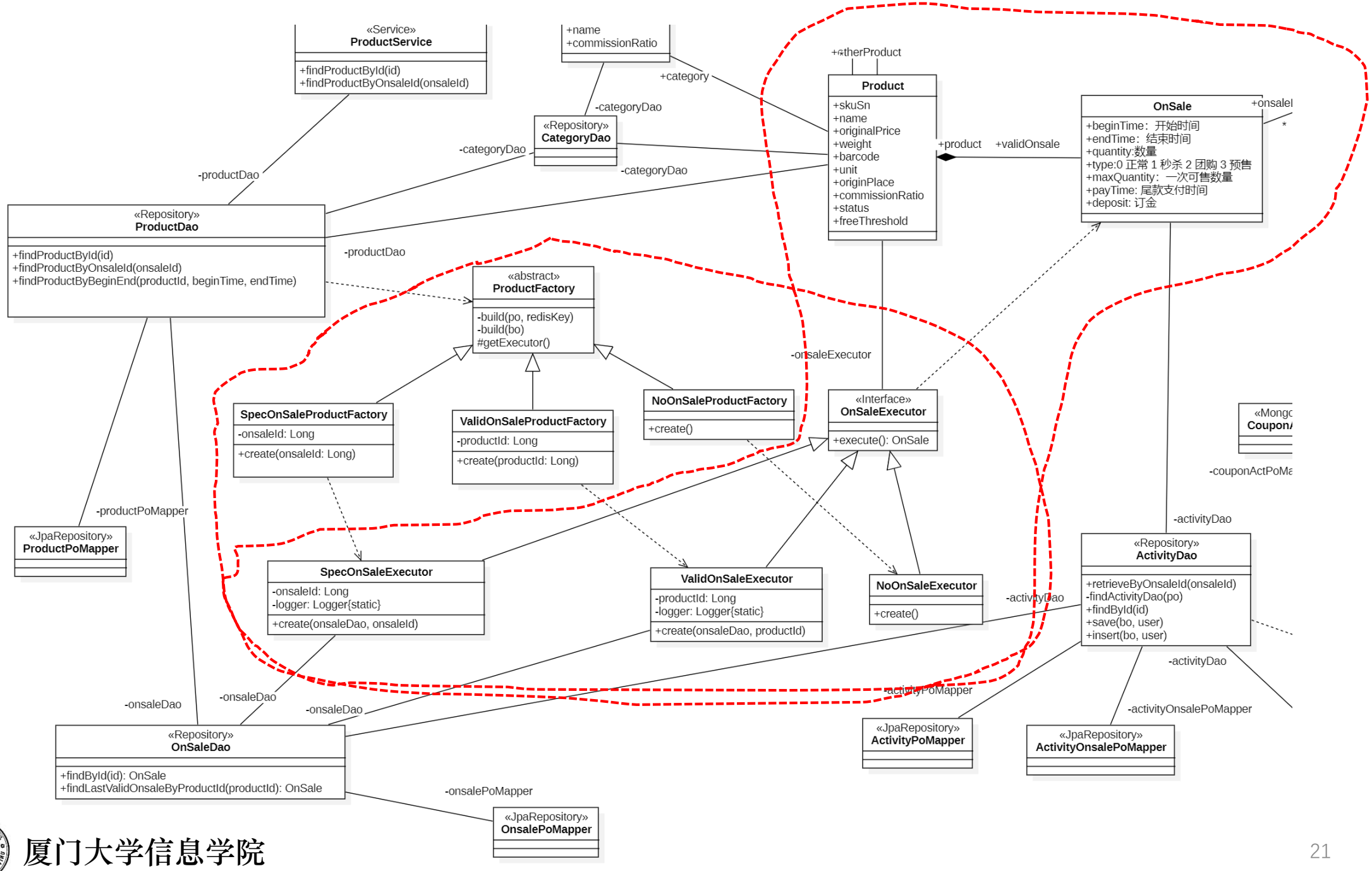
- 需要缓存
  - Category
  - Product
  - Onsale
  - Service
  - 各类Activity
  - 以及它们之间的关系



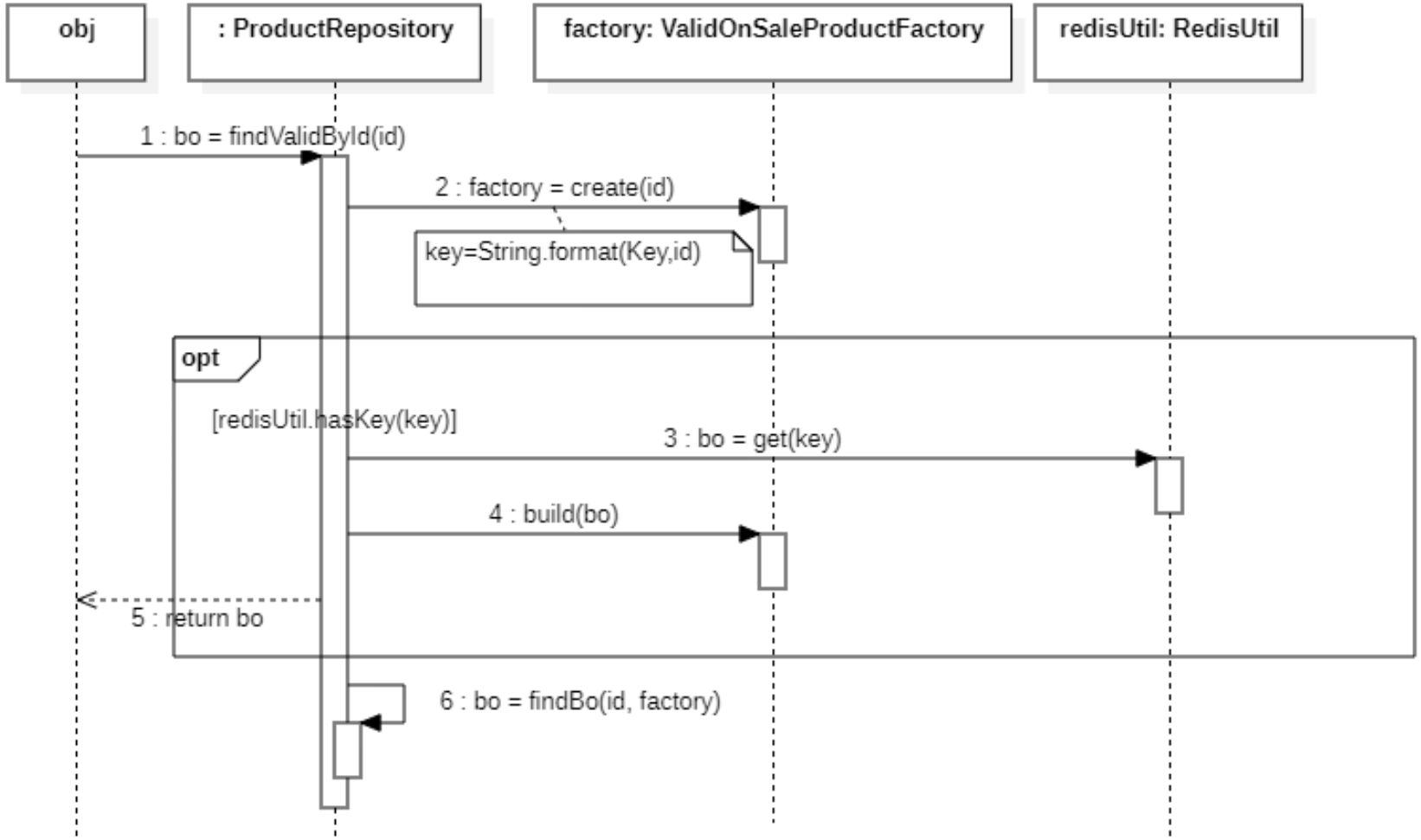
2.1 顾客查看商品详细信息

- 命令设计模式（Command）
  - 将一个请求封装为一个对象，从而使你可用不同的请求对客户进行参数化；
    - 实现发送者和接收者之间的低耦合，发送请求的对象只需要知道如何发送请求，而不必知道如何完成请求。
    - 将请求本身变成一个对象向未制定的应用对象提出请求。

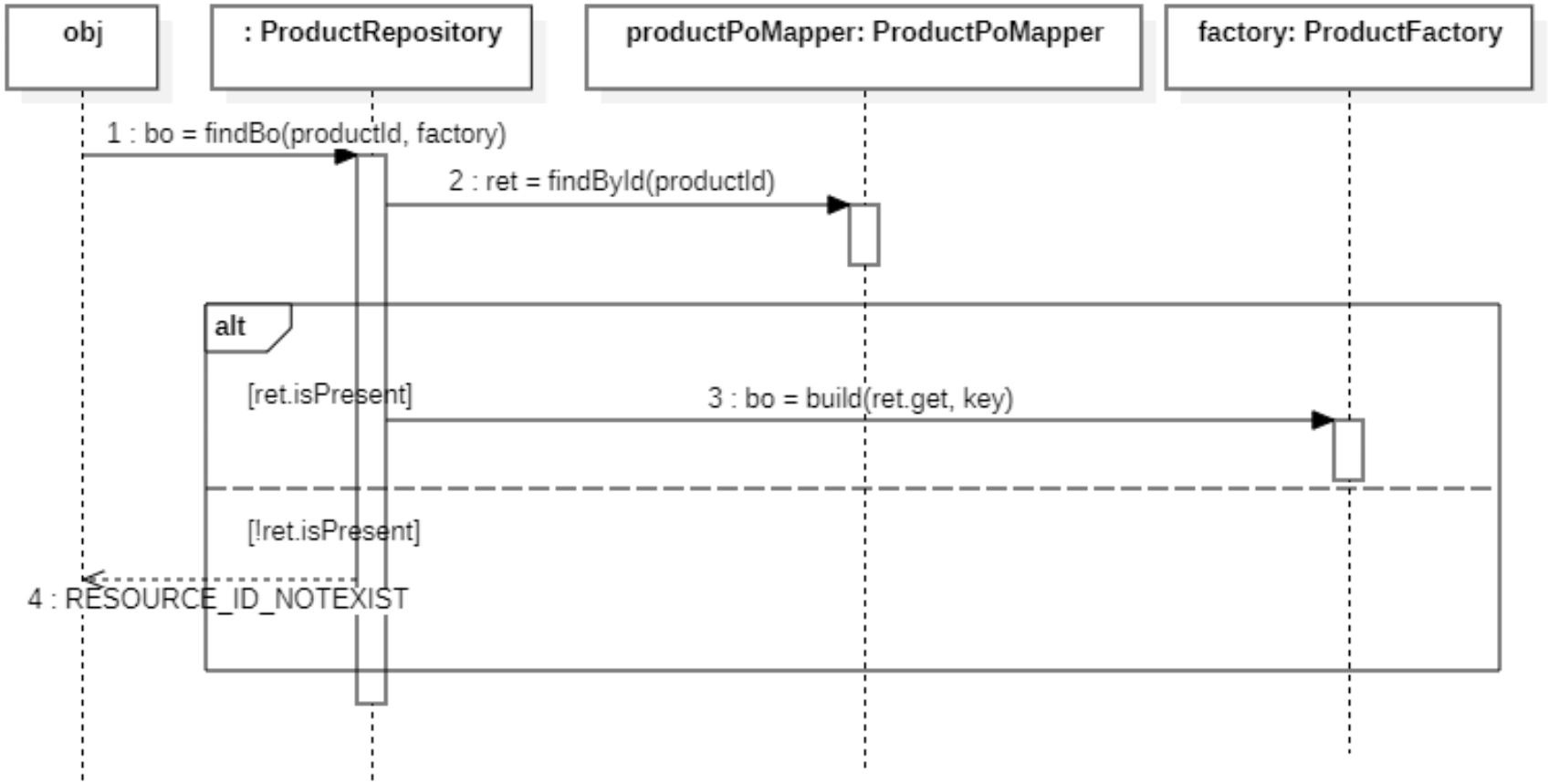



«Service»
ProductService
+findProductById(id)
+findProductByOnsaleId(onsaleId)
+name
+commissionRatio
+category
-categoryDao
«Repository»
CategoryDao
-categoryDao
-categoryDao
-productDao
«Repository»
ProductDao
+findProductById(id)
+findProductByOnsaleId(onsaleId)
+findProductByBeginEnd(productId, beginTime, endTime)
-productDao
+otherProduct
Product
+skuSn
+name
+originalPrice
+weight
+barcode
+unit
+originPlace
+commissionRatio
+status
+freeThreshold
+product
+validOnsale
OnSale
+beginTime：开始时间
+endTime：结束时间
+quantity:数量
+type:0 正常 1 秒杀 2 团购 3 预售
+maxQuantity：一次可售数量
+payTime: 尾款支付时间
+deposit: 订金
«abstract»
ProductFactory
-build(po, redisKey)
-build(bo)
#getExecutor()
-onsaleExecutor
SpecOnSaleProductFactory
-onsaleId: Long
+create(onsaleId: Long)
ValidOnSaleProductFactory
-productId: Long
+create(productId: Long)
NoOnSaleProductFactory
+create()
«Interface»
OnSaleExecutor
+execute(): OnSale
-productPoMapper
«JpaRepository»
ProductPoMapper
SpecOnSaleExecutor
-onsaleId: Long
-logger: Logger{static}
+create(onsaleDao, onsaleId)
ValidOnSaleExecutor
-productId: Long
-logger: Logger{static}
+create(onsaleDao, productId)
NoOnSaleExecutor
+create()
-activityDao
«Repository»
ActivityDao
+retrieveByOnsaleId(onsaleId)
-findActivityDao(po)
+findById(id)
+save(bo, user)
+insert(bo, user)
-activityDao
-activityPoMapper
-activityOnsalePoMapper
-onsaleDao
-onsaleDao
-onsaleDao
«Repository»
OnSaleDao
+findById(id): OnSale
+findLastValidOnsaleByProductId(productId): OnSale
«JpaRepository»
ActivityPoMapper
«JpaRepository»
ActivityOnsalePoMapper
-onsalePoMapper
«JpaRepository»
OnsalePoMapper

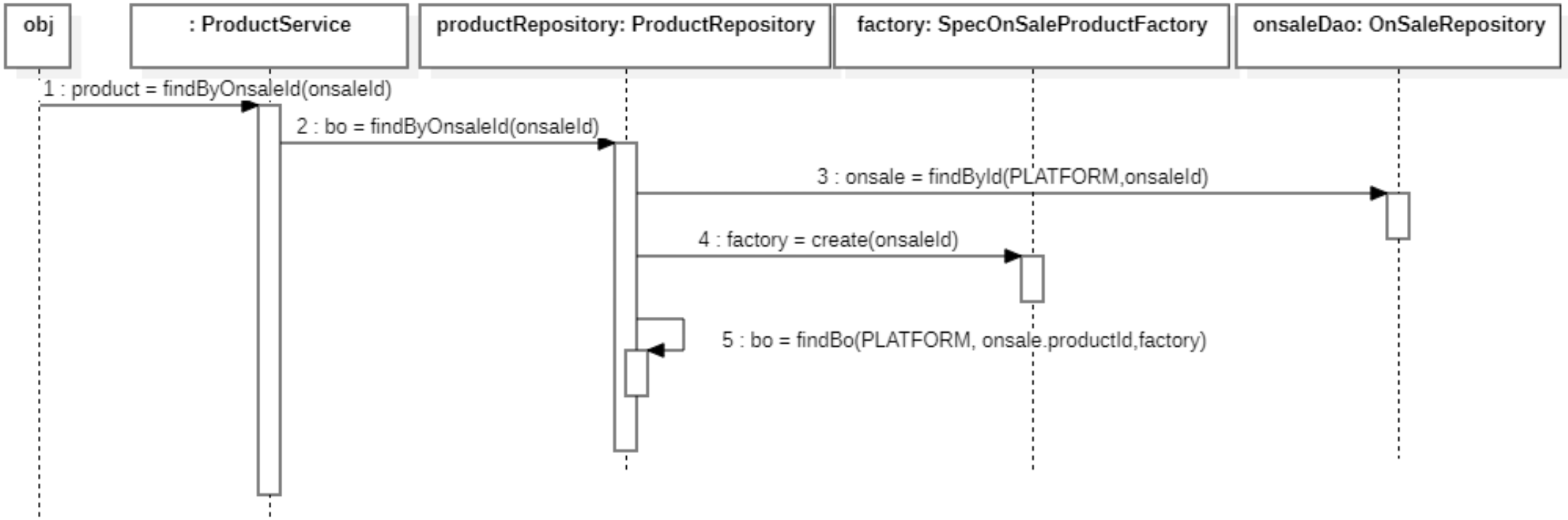
2.1 顾客查看商品详细信息



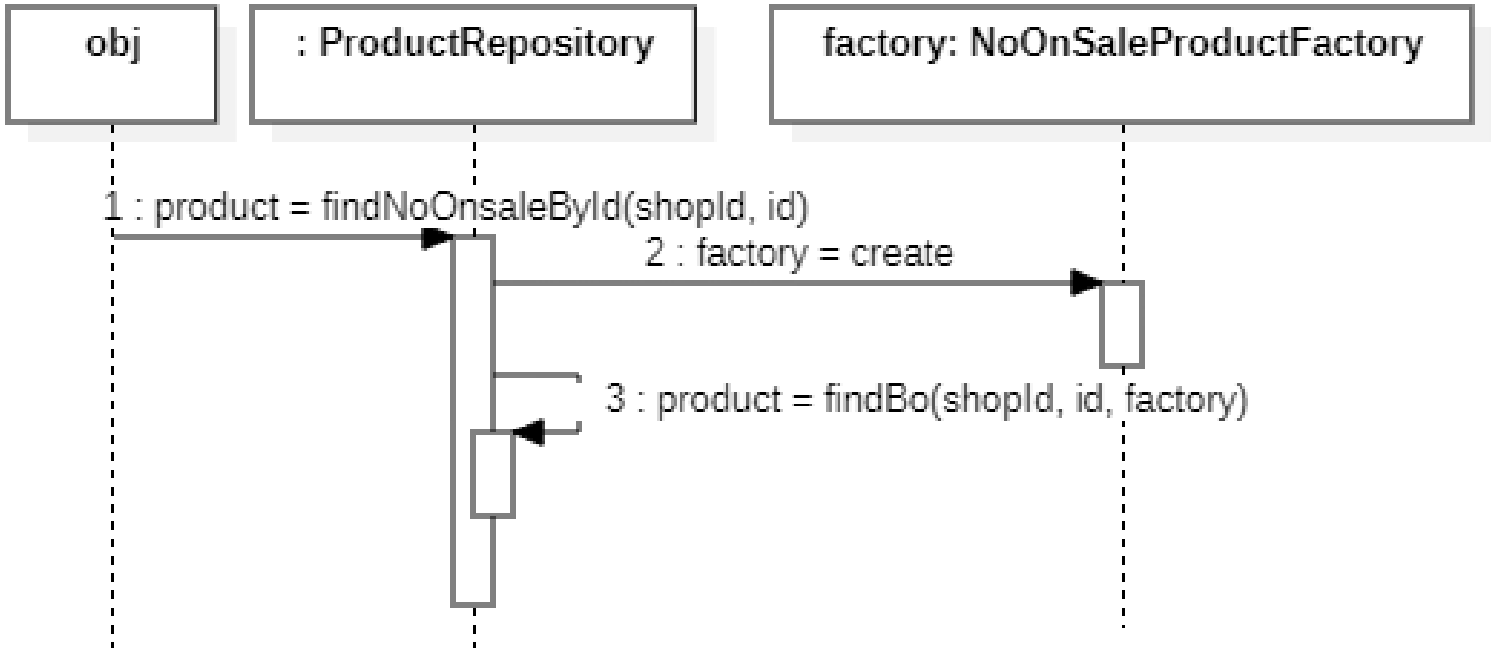
2.1 顾客查看商品详细信息



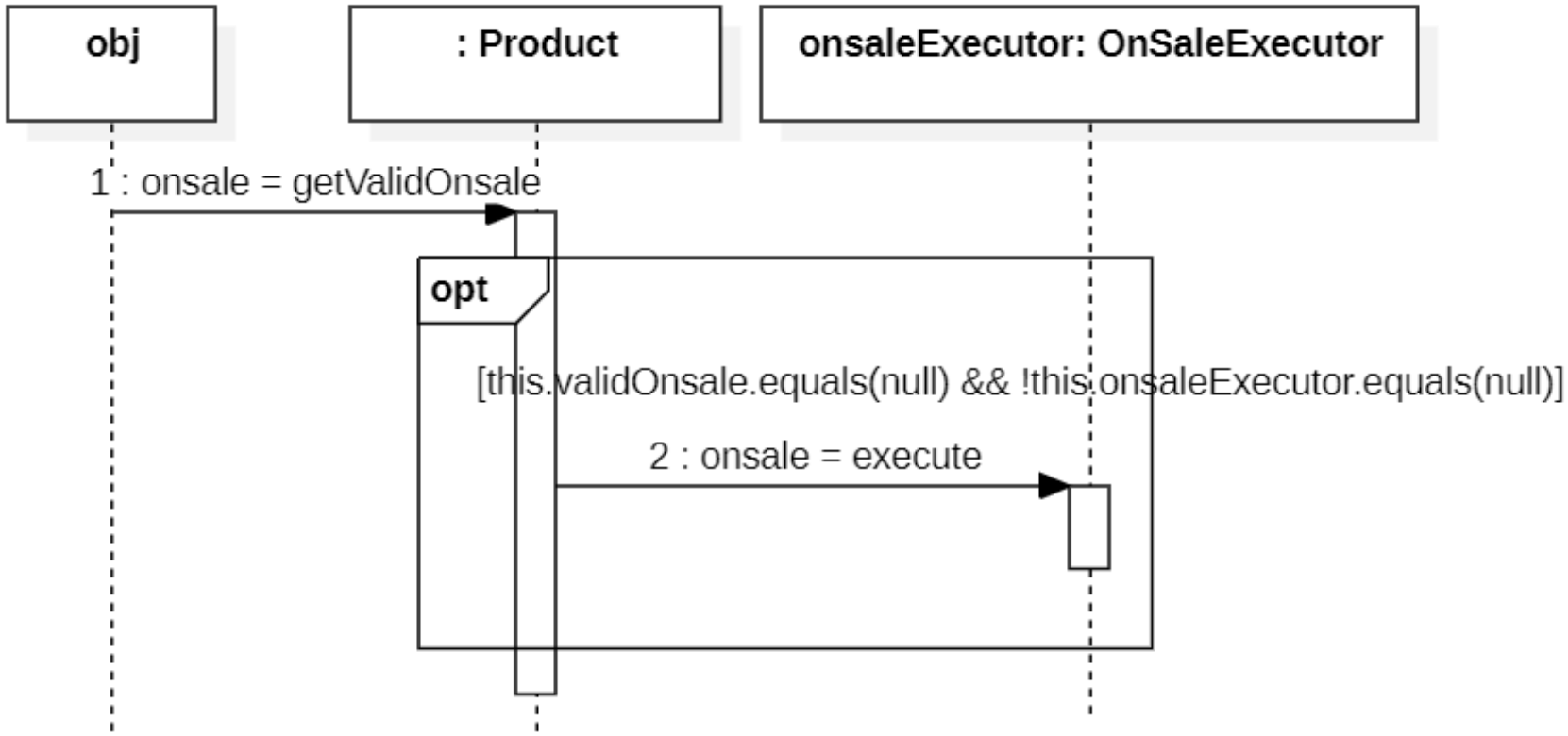
2.1 顾客查看商品详细信息



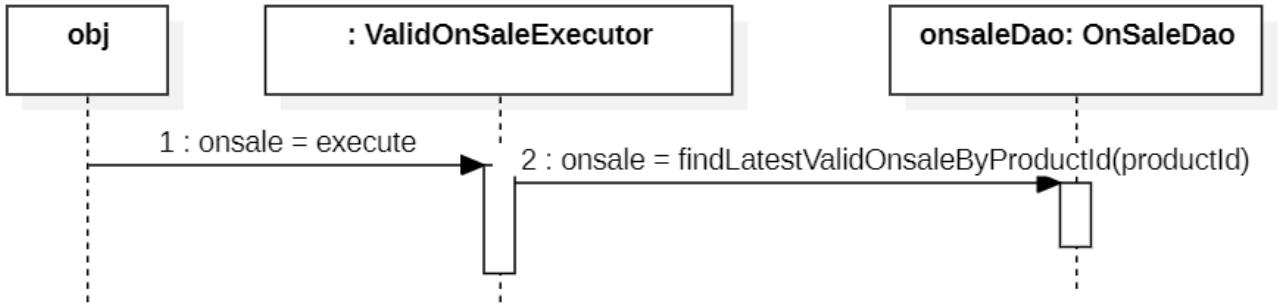
2.1 顾客查看商品详细信息

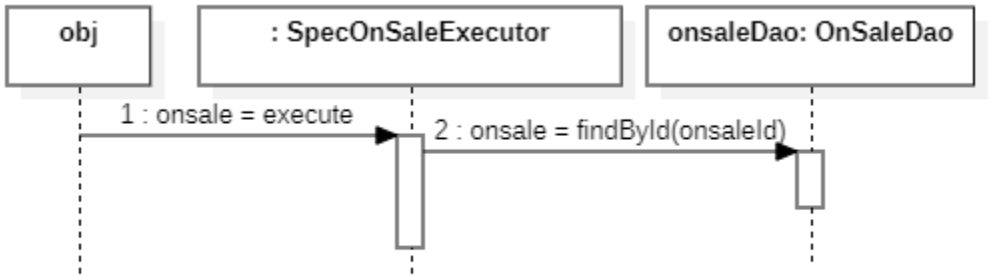


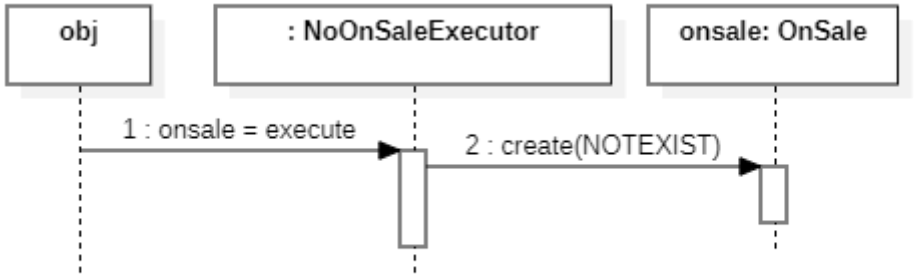
2.1 顾客查看商品详细信息



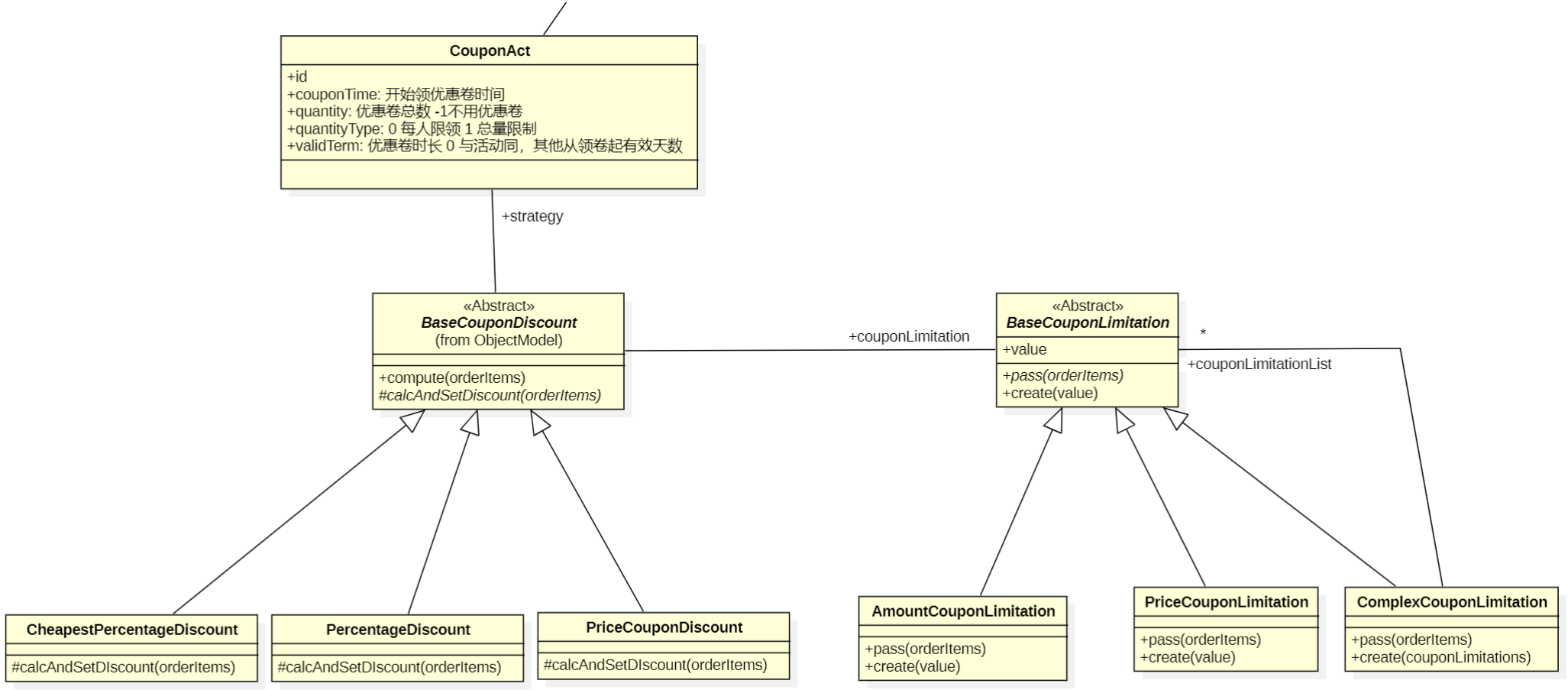
2.1 顾客查看商品详细信息





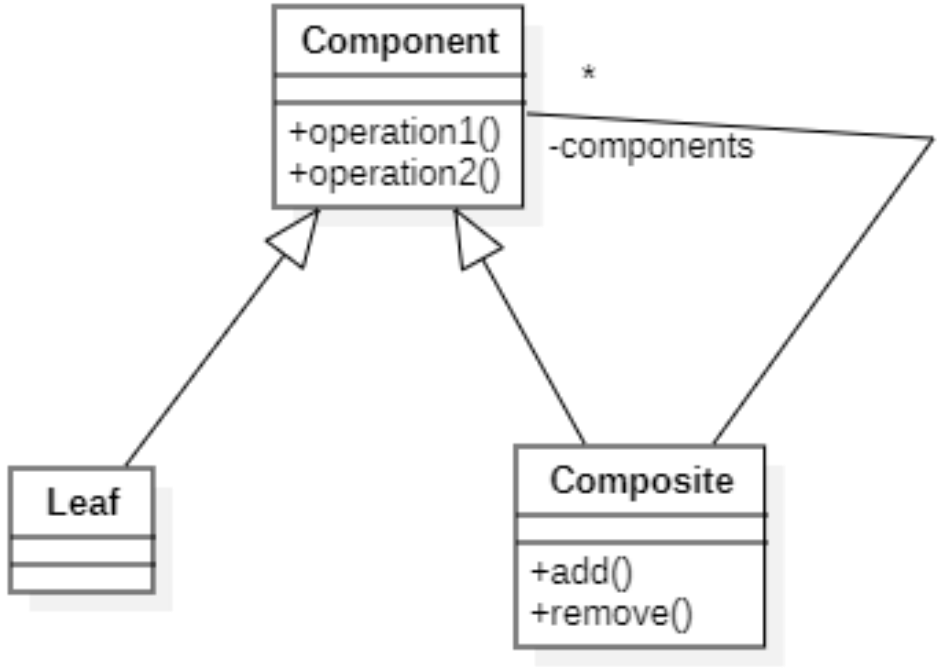


2.2 计算优惠

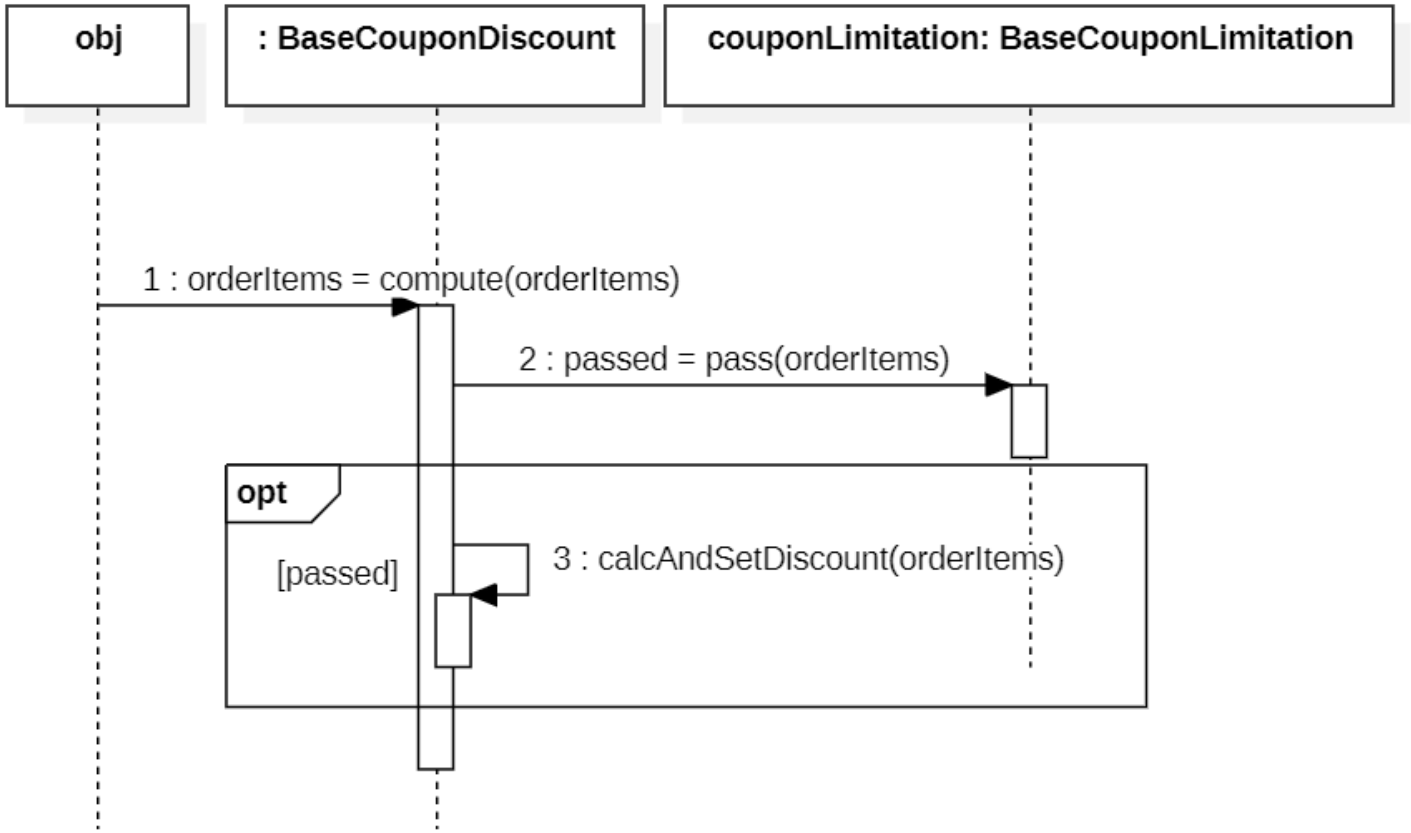

CouponAct
+id
+couponTime: 开始领优惠卷时间
+quantity: 优惠卷总数 -1不用优惠卷
+quantityType: 0 每人限领 1 总量限制
+validTerm: 优惠卷时长 0 与活动同，其他从领卷起有效天数
+strategy
«Abstract»
BaseCouponDiscount
(from ObjectModel)
+compute(orderItems)
#calcAndSetDiscount(orderItems)
+couponLimitation
«Abstract»
BaseCouponLimitation
+value
+pass(orderItems)
+create(value)
*
+couponLimitationList
CheapestPercentageDiscount
#calcAndSetDIscount(orderItems)
PercentageDiscount
#calcAndSetDIscount(orderItems)
PriceCouponDiscount
#calcAndSetDIscount(orderItems)
AmountCouponLimitation
+pass(orderItems)
+create(value)
PriceCouponLimitation
+pass(orderItems)
+create(value)
ComplexCouponLimitation
+pass(orderItems)
+create(couponLimitations)

2.2 计算优惠

- 组合设计模式(Composite)
  - 将对象组合成树形结构，使得调用者对单个对象和组合对象的使用具有一致性。



2.2 计算优惠


obj
: BaseCouponDiscount
couponLimitation: BaseCouponLimitation
1 : orderItems = compute(orderItems)
2 : passed = pass(orderItems)
opt
[passed]
3 : calcAndSetDiscount(orderItems)